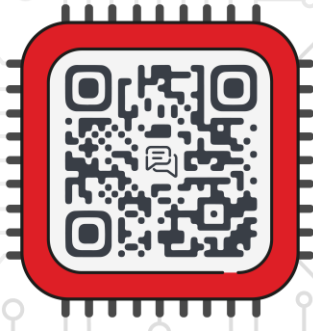




15

1

2250 sp

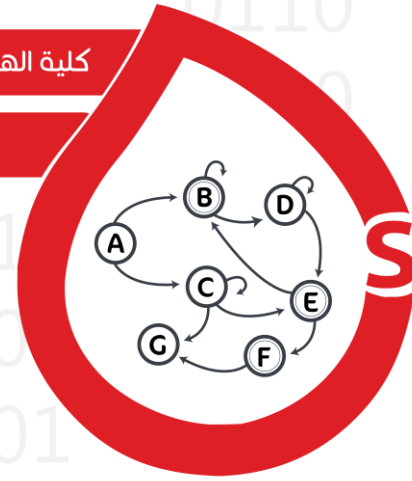
نظري

كلية الهندسة المعلوماتية

السنة الرابعة

The core of intelligent search

د. باسل الخطيب



RB Informatics; 5/10/2025

المقدمة:

مع بداية فصل دراسي جديد على بركة الله، نخطو أولى خطواتنا في عالم الذكاء الاصطناعي، وتحديدًا في مادة خوارزميات البحث الذكية، القلب النابض لأي نظام ذكي. في هذه المادة لن نفهم الشيفرة للخوارزميات فقط، بل نهدف لفهم المنطق الكامن من وراءها، لنتمكن من تعديلها وتصميمها لمواجهة أصعب التحديات في مجالات مختلفة كالتخطيط، واتخاذ القرار، والروبوتات. هيا بنا نتعلم كيف نجعل الآلة تفكر بذكاء.

فائدة (مصطلحات أساسية بالذكاء الاصطناعي):

1. البحث (Search):

جعل الذكاء الاصطناعي قادراً على البحث عن حلول لمشكلة ما سواء كانت المشكلة محاولة معرفة الطريق من أ إلى ب، أو معرفة كيفية لعب لعبة ما.

2. المعرفة (Knowledge):

جعل الذكاء الاصطناعي قادراً على معرفة المعلومات واستخلاص استنتاجات من هذه المعلومات

$$\begin{array}{c} P \rightarrow Q \\ P \\ \hline Q \end{array}$$

3. عدم اليقين (Uncertainty):

معرفة كيفية تعامل الكمبيوتر مع الأحداث غير المؤكدة (الاستدلال الاحتمالي).

4. التحسين (Optimization):

جعل الجهاز يختار أفضل طريقة لحل المشكلة .

5. التعلم (Learning):

جعل الكمبيوتر قادراً على التعلم من البيانات ومن ثم القدرة على أداء مهمة بشكل أفضل بناءً على هذه المعرفة بالبيانات والتجارب، مثلاً عملية تصنيف البريد الإلكتروني إلى هام أو غير مرغوب به.



6. الشبكات العصبية (Neural Networks):

جعل الذكاء الاصطناعي قادراً على الاستلهام من الذكاء البشري.

7. اللغة (Language):

محاولة جعل الكمبيوتر قادراً على فهم اللغة البشرية.

خوارزميات البحث (Search Algorithm):

تقوم خوارزميات البحث في البحث عن حل لمشكلة ما من خلال استكشاف المسارات (paths) أو الخيارات الممكنة (Options) في فضاء المشكلة أو المسألة لدينا، ويستخدم الذكاء الاصطناعي خوارزميات البحث في مهام مثل:

- إيجاد أفضل طريق للانتقال من A إلى B في الخرائط.

- تحديد سلسلة الحركات (moves) للعبة ما

فالخوارزمية: هي مجموعة تعليمات محددة لحل المسألة

والبحث: هو عملية استكشاف منظّمة لمجموعة من الاحتمالات للوصول إلى الهدف.

خوارزميات البحث لها بعدين رئيسيين:

1. المعرفة المتوفرة لدينا (Knowledge)

- البحث دون معرفة (Uniformed search) أو البحث الأعمى (Blind Search):

تعريف: هي استراتيجية لا يتم فيها استخدام أي نوع من المعلومات الخاصة بالمهمة، أي لا توجد تخمينات ذكية (Heuristic) للوصول إلى الهدف. تهتم بالحالة الحالية وتتحقق مما إذا كانت هي الهدف أم لا.

أمثلة: DFS, BFS, Uniform Cost Search (سنشرح هذه الخوارزميات لاحقاً)

- البحث مع وجود معرفة سابقة (Informed Search):

تعريف: يطلق عليه أيضاً البحث الإرشادي (heuristic search) هذا النوع لديه وصول إلى معلومات خاصة بالمهمة، مما يجعله أكثر كفاءة في توجيه عملية البحث نحو الهدف.

أمثلة: A* search algorithm, greedy best first search

مثال لتوضيح الفرق بين العمياء والإرشادية:

لنتخيل أن المسألة تشبه متاهة ضخمة والبحث هو الطريقة التي نستخدمها لنقرر إلى أين نتحرك لاحقاً للخروج من المتاهة.

في الخوارزمية العمياء نتحرك في كل الاتجاهات بدون معرفة مسبقة للوصول إلى الهدف وهو الخروج من المتاهة.

أما في الخوارزمية المستنيرة أو الإرشادية، فنحن نمتلك فكرة عن الاتجاه الأفضل الذي يجب أن نسلكه للوصول إلى الهدف.

2. جودة الحل (Quality)

- Any path searches: البحث عن أي حل ممكن يوصلنا للحالة الهدف
- optimal searches: البحث عن الحل الأمثل وهو أفضل حل مثل المسار الأقصر أو الأقل تكلفة.

Graph

وبعد ان تعرفنا على أهم المصطلحات في خوارزميات البحث وأنواعها، دعونا نتكلم عن كيف يتم تمثيل مسائل البحث الذكية.

في الذكاء الاصطناعي، تمثل أي مسألة بحث على شكل Graph بحيث

- العقد (Nodes) تمثل الحالات (States)
- الحواف (Edges) تمثل الأفعال أو الإجراءات (Actions) التي تؤدي إلى الانتقال من حالة إلى أخرى.

أمثلة:

1. الخرائط: (Maps)

- كل مدينة تمثل عقدة.
- كل طريق يمثل حافة تربط بين عقدتين.

2. شبكات الإنترنت: (Networks)

- كل موقع يمثل عقدة.
- الروابط بين المواقع تمثل الحواف.

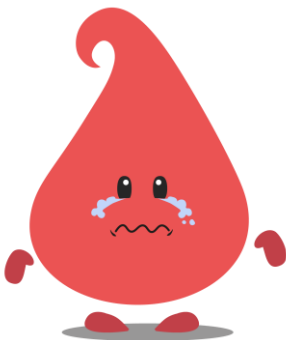
3. لعبة الشطرنج: (Chess Game)

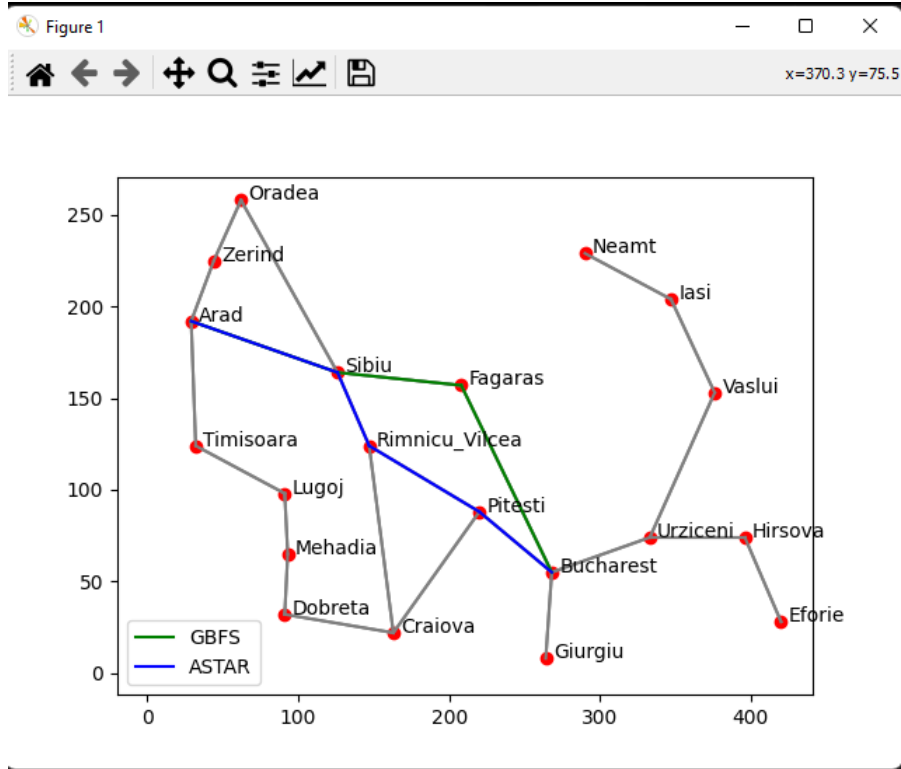
- كل وضعية للقطع على اللوح تمثل عقدة.
- كل حركة ممكنة تمثل حافة تربط بين وضعيتين.

مثال عملي

خريطة مدن رومانيا (Romania Map) وهي تستخدم كثيرًا في مسائل البحث، حيث:

- المدن تمثل العقد (Nodes)
- الطرق بين المدن تمثل الحواف أو الوصلات (Edges)
- قيمة كل حافة هي تكلفة الانتقال وهي فرضاً المسافة بالكيلومتر
- المطلوب: نريد الانطلاق من مدينة Arad والوصول إلى Bucharest
- الهدف: تطبيق خوارزميات البحث لإيجاد أقصر طريق بين المدينتين.





صيغة المشكلة - Formulating the problem

لتطبيق أي خوارزمية بحث، يجب أولاً تحديد ثلاثة عناصر أساسية:

1. Formulating the goal - تحديد الهدف

الهدف في المثال السابق هو الوصول إلى مدينة Bucharest

2. Formulate Problem - تحديد المشكلة

- تحديد الحالات (States) وهي تمثل المدن
- تحديد الأفعال (Actions) هي تمثل القيادة بين المدن

3. Find Solution - إيجاد الحل

يتمثل الحل في تحديد تسلسل المدن الذي يكون المسار الصحيح للوصول إلى الهدف بأقل تكلفة ممكنة، وهو هنا:

Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bucharest

بينما الحل العادي (لا يشترط أن يكون الأقل تكلفة):

Arad → Sibiu → Fagaras → Bucharest



كُن لِنَفْسِكَ كُلَّ شَيْءٍ

تمثيل المشكلة (Representing the Problem):

الفكرة الرئيسية في الذكاء الاصطناعي الكلاسيكي هي البحث في فضاء الحالات عن تسلسل من الأفعال يؤدي إلى الهدف. أي أننا نمثل المشكلة كـ (Graph)، ونستخدم خوارزمية بحث معينة لاكتشاف المسار الصحيح للوصول إلى الهدف. لفعل ذلك يجب معرفة مجموعة من المفاهيم الهامة، وسوف نقوم باستخدام عدة أمثلة لشرح كيفية تمثيل المشكلة وتطبيق خطوات البحث عليها.

1. The 8-Puzzles

هي رقعة مؤلفة من مربع أبعاده 3×3 تحتوي على ثمانية أرقام ومربع فارغ. الهدف: ترتيب الأرقام وجعل الفراغ في النهاية.

1. Agent (الوكيل):

الكائن الذي يحل المشكلة، وهو الخوارزمية الذكية التي تقوم على حل اللغز.

2. State (الحالة):

تمثل حالة المسألة في مرحلة ما، وهي توزيع البلاطات في اللحظة الحالية. كل ترتيب جديد للبلاطات يمثل حالة مختلفة.

3	8	1
2	5	4
6	7	

3	2	1
7	5	6
4	8	

1	2	3
4	5	6
7	8	

الحالة الهدف

3. Goal State (الحالة الهدف):

وهي الحالة النهائية التي تمثل حل اللغز

1	2	3
4	5	6
7	8	



4. initial state (الحالة الابتدائية)

الحالة الابتدائية في المسألة، وهي التي نبدأ منها محاولة البحث عن الحل

5. action/operators (الأفعال):

مجموعة من الأفعال أو الخطوات التي يمكن تطبيقها على الحالة الحالية

1	2	3
4	6	7
5	8	


 Move_blank_right
 تحريك الفراغ لليمين


 Move_blank_left
 تحريك الفراغ لليسار


 Move_blank_up
 تحريك الفراغ للأعلى


 Move_blank_down
 تحريك الفراغ للأسفل

فـالـ actions تختلف من حالة لحالة أخرى بحسب موقع الفراغ

6. Transition model (تابع الانتقال):

الـ actions والـ state ترتبط مع بعضها البعض عن طريق الـ transition model وهي تتمثل بتابع Result (s,a) يعيد الحالة الناتجة عن تطبيق الفعل a على الحالة s.

Result (

1	2	3
4	6	7
5	8	

,

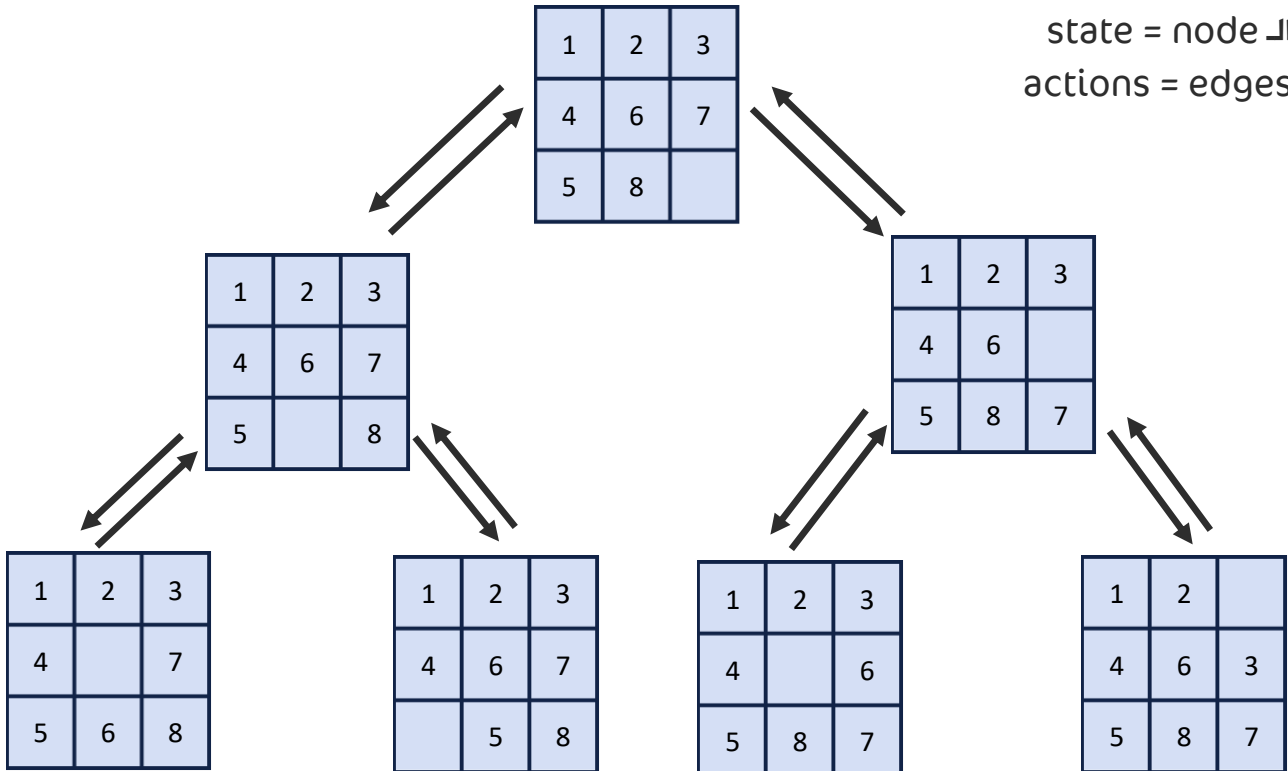
) =

1	2	3
4	6	
5	8	7

7. state space

هو مجموعة جميع الحالات التي يمكن الوصول لها من الحالة الابتدائية بتطبيق مجموعة من الأفعال
يتم تمثيل فضاء الحالات بالـ graph حيث:

الـ state = node
الـ actions = edges



8. Goal Test

هي طريقة لتحديد ما إذا كانت الحالة التي وصلنا إليها هي الحالة الهدف أم لا. أهميتها أنها تجعل عملية البحث محدودة، فلا يتم استكشاف جميع الحالات، بل فقط الحالات التي قد تؤدي إلى تحقيق الهدف. وقد يكون الهدف عاماً مثل:

"أريد الوصول إلى أي مدينة في ألمانيا"

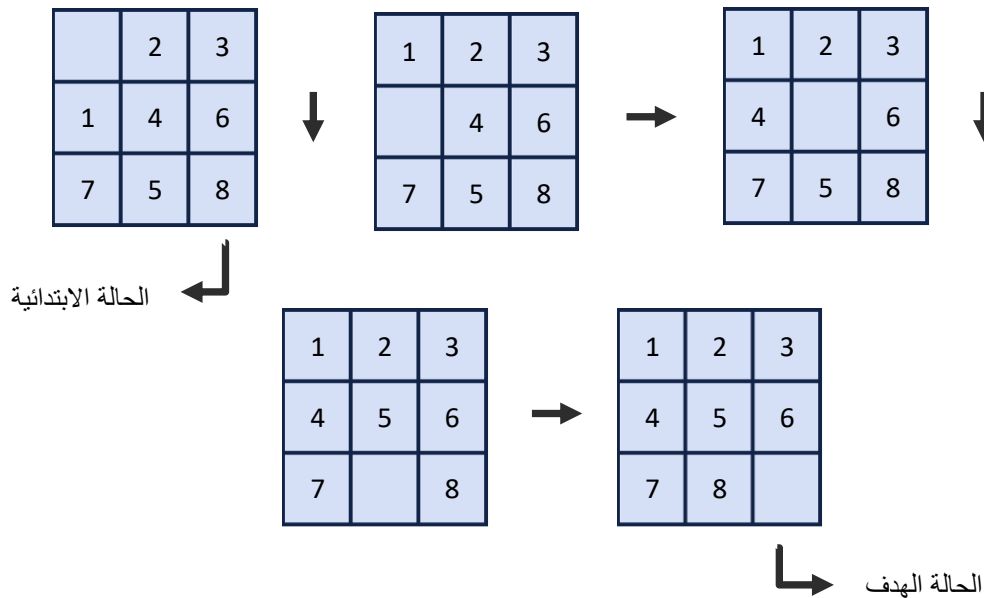
goal test () = هل المدينة الحالية تقع ضمن ألمانيا
محددًا مثل:

"أريد الوصول إلى مدينة Bucharest"

goal test () = هل الموقع الحالي هو Bucharest

9. Path Cost (تكلفة الطريق):

هو مجموع التكاليف الناتجة عن الأفعال التي قمنا بها للوصول من الحالة الابتدائية إلى الحالة الهدف أي أنه يجيب على سؤال: "كم كلفنا الوصول إلى هنا من الحالة الابتدائية؟"



إذا فرضنا أن تكلفة كل action هي 1 فيكون تكلفة الطريق (Path cost) يساوي 4



كل نجاح يبدأ بخطوة

10. Representing the state (تمثيل الحالة)

■ إما مصفوفة ثنائية:

{ 1 , 3 , 2 },

{ 7 , 8 , 6 },

{ 5 , 4 , # }

حيث # يمثل الفراغ

1	3	2
7	8	6
5	4	

■ أو مصفوفة أحادية:

[1,3,2,7,8,6,5,4,#]

لمعرفة موقع الرقم X في المصفوفة الأحادية:

نفرض أن X يقع في العمود z والسطر i في اللعبة وعدد الأعمدة الكلي هو col

فيكون موقع X في المصفوفة الأحادية هو:

$$i * cols + z$$

تمثيل موقع كل رقم على اللوحة:

1 -> (1, 1)

4 -> (3, 2)

7 -> (2, 1)

2 -> (1, 3)

5 -> (3, 1)

8 -> (2, 2)

3 -> (1, 2)

6 -> (2, 3)

-> (3, 3)

نختار الطريقة التي تجعل العمليات أسهل ما يكون، وفي حالتنا هذه فإن المصفوفة الثنائية هي أحسن تمثيل

لأنها تسهل عملية الانتقال من حالة إلى أخرى ومعرفة قيمة أي خانة

ملاحظة: لكل operator يوجد precondition أي شرط مسبق يحدد هل يمكن استخدام هذا الـ action

أم لا

Specify the operators/actions ←

يمكن تمثيل الأفعال بطرق مختلفة



اسع ودع النتائج تأتي وحدها

الطريقة الثانية بحسب الفراغ

Move blank left
Move blank right
Move blank up
Move blank down

هذه الطريقة أبسط وأكثر فعالية لأن كل خطوة تعتمد فقط على موقع الفراغ ويكون عدد الحركات الممكنة هو 4 حركات فقط.
فالـ actions تختلف من حالة لحالة أخرى بحسب موقع الفراغ

الطريقة الأولى بحسب الرقم

Move number 1 left
Move number 1 right
Move number 1 up
Move number 1 down
Move number 2 left
Move number 2 right
Move number 2 up
Move number 2 down

....
هذه الطريقة تنتج عدد كبيراً من العمليات (لكل رقم ولكل اتجاه) حيث يوجد ثمانية أرقام كل رقم يمكن أن يتحرك بـ 4 اتجاهات فيصبح عدد الحركات الممكنة هو 32 حركة وهذا عدد كبير

Formulation of a search problem

إنّ أي مسألة بحث يمكن صياغتها بما يلي:

Initial state ←

Actions ←

Transition model ←

Goal test ←

Path cost ←

2. Farmer, wolf, goat, cabbage (FWGC)

الهدف هو نقل الفلاح مع الذئب والمعزة والملفوف من الضفة النهر اليسرى إلى الضفة اليمنى باستخدام قارب لا يحمل سوى الفلاح وشيء واحد في كل مرة.

1. الحالة الابتدائية:

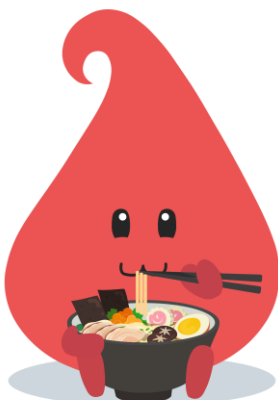
(left, left, left, left) = (F, W, G, C)

أي أن الجميع في الضفة اليسرى من النهر.

2. الهدف:

(right, right, right, right) = (F, W, G, C)

أي أن الجميع عبروا إلى الضفة اليمنى من النهر.



3. العمليات:

- Move F الفلاح يعبر الضفة وحده
- Move F with W الفلاح يعبر ومعه الذئب
- Move F with G الفلاح يعبر ومعه المعزة
- Move F with C الفلاح يعبر ومعه الملفوف

4. القيود:

- لا يمكن ترك الذئب والمعزة وحدهما دون الفلاح.
- لا يمكن ترك المعزة والملفوف وحدهما دون الفلاح.

5. الحل باستخدام ال BFS (سنتعرف عليه لاحقاً)

- 1- F + G → Right
- 2- F → Left
- 3- F + W → Right
- 4- F + G → Left
- 5- F+C → Right
- 6- F → Left
- 7- F+G → Right

3. Mathematical equation example

$$X = (X + X) \cdot \frac{1}{2}$$

الهدف هو إثبات أن المعادلة صحيحة رياضياً باستخدام قوانين الجبر المنطقي.

1. الحالة الابتدائية:

$$X = (X + X) \cdot \frac{1}{2}$$

2. الحالة الهدف:

$$X = X$$

3. العمليات:

التوزيع:

$$\frac{1}{2}(X + X) = \frac{1}{2}X + \frac{1}{2}X$$

التبسيط:

$$\frac{1}{2}X + \frac{1}{2}X = \frac{2}{2}X$$

الاختزال:

$$X = X$$

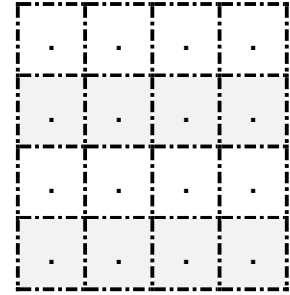


4. The 4 Queens Problem

الهدف هو وضع 4 ملكات على رقعة شطرنج بحجم 4x4 بحيث لا تهدد أي ملكة الأخرى.
تهدد الملكة: إذا كانت مع ملكة ثانية في نفس الصف أو العمود أو على نفس القطر.

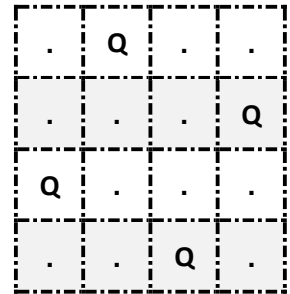
4. الحالة الابتدائية:

لوح فارغ



5. الهدف:

لوح يحتوي على 4 ملكات حيث لا توجد ملكتان في نفس الصف أو العمود أو القطر.



6. العمليات:

- وضع الملكة في خانة حيث لا تهاجم أي ملكة موجودة مسبقاً.
- إذا حدث تعارض، احذف الملكة وعد خطوة للخلف (Backtracking)

تذكرة: الـ Backtracking

أسلوب بحث يستخدم في المسائل التي تتطلب تجربة جميع الاحتمالات الممكنة، فإذا قمنا بسلك طريق أو خطوة غير صحيحة أو صالحة، نتراجع خطوة إلى الوراء ونجرب خياراً آخر.

لا تدع الماضي يقف في

طريق المستقبل



7. آلية عمل الخوارزمية:

1. نبدأ من الصف الأول

2. نحاول وضع الملكة في كل مكان ممكن.

3. بعد وضع الملكة نتحقق:

هل الوضع هذا آمن؟ (أي لا يوجد ملكة ثانية في نفس الصف أو العمود أو القطر)

نعم ←

ننتقل للصف التالي ونكرر

لا ←

نتراجع backtrack إلى الصف السابق ونغير موقع الملكة هناك

4. نكرر حتى نضع جميع الملكات أو نجرب جميع الحالات الممكنة..

5. The water jug problem

لدينا دلوّان (jugs) الأول سعته 4 وحدات، والثاني سعته 3 وحدات والمطلوب جعل أحد الدلوين يحتوي على وحدتين من الماء

القيود:

- يمكننا إما تعبئة أحد الدلوين أو كلاهما بالكامل من الصنبور
- إفراغ أحد الدلوين أو كلاهما بشكل كامل
- نقل محتوى أحد الدلوين إلى الدلو الآخر (نقل عدد صحيح من الوحدات)

تحليل المسألة:

1. كل حالة (State) يمكن تمثيلها بزوج من القيم (x, y) حيث:

x : كمية الماء في الدلو الكبير (سعته 4 وحدات)

y : كمية الماء في الدلو الصغير (سعته 3 وحدات)

2. الحالة الابتدائية:

$(0, 0)$ كلا الدلوين فارغ

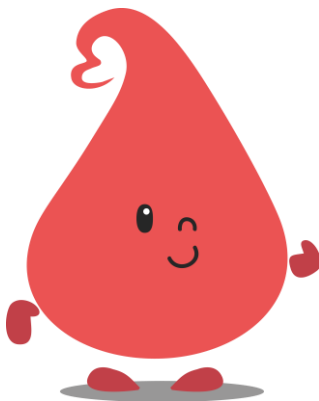
3. الحالة الهدف:

إما $(n, 2)$ أو $(2, n)$ أي أن أحدهما يحتوي على 2 لتر من الماء

4. الـ Actions:

ملئ، إفراغ، سكب من دلو إلى آخر

وبالتالي سيكون لدينا ستة أنواع رئيسية من الأفعال:



- ملئ الدلو الكبير من الصنبور
 $(x, y) \leftarrow (4, y)$
- ملئ الدلو الصغير من الصنبور
 $(x, y) \leftarrow (x, 3)$
- إفراغ الدلو الكبير
 $(x, y) \leftarrow (0, y)$
- إفراغ الدلو الصغير
 $(x, y) \leftarrow (x, 0)$
- سكب من الكبير إلى الصغير حتى يمتلئ
 $(x, y) \leftarrow (x - k, y + k)$
حيث $k = \min(x, 3 - y)$
- سكب من الصغير إلى الكبير حتى يمتلئ
 $(x, y) \leftarrow (x + k, y - k)$
حيث $k = \min(4 - x, y)$

5. الحل باستخدام التفكير المنطقي :
نبدأ من الحالة (0, 0)

1. املاُ الدلو الصغير $(0, 3) \leftarrow$

2. اسكب من الصغير إلى الكبير $(3, 0) \leftarrow$

3. املاُ الدلو الصغير مرة أخرى $(3, 3) \leftarrow$

4. اسكب من الصغير إلى الكبير حتى يمتلئ $(4, 2) \leftarrow$

5. أفرغ الدلو الكبير $(0, 2) \leftarrow$

الهدف تحقق، الآن يوجد 2 وحدة من الماء في الدلو الصغير

ملاحظة: k تمثل كمية الماء المنقولة من دلو إلى آخر

لا يمكن نقل كمية من الماء أكبر من الكمية الموجودة في الدلو المراد النقل منه، ولا يمكن نقل كمية

أكبر مما يتسع له الدلو الآخر، وهذا يشرح مبدأ الشرطين:

$$k = \min(x, 3 - y) \text{ و } k = \min(4 - x, y)$$

لدينا غرفتان A و B، كل غرفة قد تكون نظيفة أو متسخة، المكنسة وهي الـ agent قد تكون في A أو B هدف المكنسة هو تنظيف الغرفتين

1. العمليات المتاحة (Actions):

- تنظيف المكان (Suck) ينظف الغرفة الموجود بها
- Move right أي الانتقال من A إلى B
- Move left أي الانتقال من B إلى A

2. عدد الحالات الممكنة:

- موقع المكنسة = 2 ← إما في A أو B
 - حالة الغرفة A = 2 ← نظيفة أو متسخة
 - حالة الغرفة B = 2 ← نظيفة أو متسخة
- فيكون عدد الحالات الكلي هو $2 \times 2 \times 2 = 8$

3. الحالة الهدف

إما (A, Clean, Clean) أو (B, Clean, Clean)

4. الـ Transition model:

- ← (A, Dirty, Dirty) (A, Clean, Dirty) أي قمنا بتنظيف الغرفة A
- ← (A, Clean, Dirty) (B, Clean, Dirty) أي حركنا المكنسة من الغرفة A إلى B
- ← (B, Clean, Dirty) (B, Clean, Clean) أي قمنا بتنظيف الغرفة B

The Formal Definition of problem

1. Initial state:

الحالة التي تبدأ فيها المكنسة: (A, Dirty, Dirty)

2. Actions:

{suck, move left, move right}

3. Transition Model:

Result ((A, Dirty, Dirty), Suck) = (A, Clean, Dirty)

4. Goal test:

التحقق فيما إذا كانت الحالة الحالية هي الهدف:

goal test(s) = (B = clean) $\wedge$ (A = clean)



أي يجب ان تكون A نظيفة و B نظيفة

5. Path cost:

القيمة العددية التي تمثل تكلفة المسار

عادة نقول:

step cost = 1 لكل عملية

path cost = مجموع الخطوات التي قامت بها المكنسة وهي هنا ثلاث خطوات

**وجودك في الحياة يجب أن يكون
إضافةً لا تكراراً**

